DNSC 6330: Responsible Machine Learning*

Lecture 02: Explainability and Interpretability as Diagnostic Tools

Michael Akinwumi

Adjunct Professor of Responsible Machine Learning

Department of Decision Sciences

The George Washington University, School of Business

March 23 & 24, 2026

*This material is shared under a CC BY 4.0 license which allows for editing and redistribution, even for commercial purposes. However, any derivative work should attribute the author.

Outline (2.5 hours)

1. Recap of Lecture 01
2. Transparency & Alignment
3. Methods Overview
4. Limitations
5. Live Coding
6. Homework
7. Conclusion

Recap: The Alignment Problem in Three Layers

What we established

- Responsible ML = **defensible** decisions across the full lifecycle.
- Alignment gap: $\Delta = U(h_U^\star) - U(h_L^\star) > 0$.
- Three layers of alignment:
  1. **Mathematical**
  2. **Behavioural**
  3. **Institutional**

Cases examined

- **Word2Vec** — representation bias institutionalised at scale.
- **COMPAS** — calibration $\neq$ equal error rates; fairness definitions conflict.
- **Facial recognition** — aggregate accuracy conceals subgroup harm.

Bridge to today

If we cannot *see inside* a model, we cannot diagnose misalignment, successfully challenge it in court, or fix it in production. **Transparency is a precondition for alignment.**

Recap: COMPAS Live-Coding Results

Empirical pattern (Broward County, $n \approx 7{,}000$)

$$\widehat{\mathrm{FPR}}_{\mathrm{Black}} > \widehat{\mathrm{FPR}}_{\mathrm{White}} \tag{1.1}$$

$$\widehat{\mathrm{FNR}}_{\mathrm{White}} > \widehat{\mathrm{FNR}}_{\mathrm{Black}} \tag{1.2}$$

- Risk scores were *approximately calibrated* across groups.
- Yet error-rate parity *failed* — an instance of the **Impossibility Theorem**.
- Governance question: which errors matter most, for whom, and why?

Open question

Even if we accept the COMPAS score as calibrated, we still cannot answer:

1. *Why* did individual i receive score s?
2. What would have to change for a different score?
3. Is the model learning causal signals or proxies?

These are `transparency` questions.

The Transparency Problem

From Brian Christian's The Alignment Problem

The ability to understand what is happening inside a ML model and why it behaves as it does.

Why this matters:

- accountability
- debugging
- regulatory oversight
- trust

Without transparency:

- misalignment remains hidden
- bias becomes invisible

Transparency Does Not Equal Trust

From *Machine Learning for High-Risk Applications* (Hall & Gill, 2023), Chapter 2:

> *"Transparency enables understanding, and understanding is different from trust. Greater understanding of a poorly built ML system may actually* **decrease** *trust."*

- Transparency $\rightarrow$ understanding $\not\Rightarrow$ trust.
- **Actionable recourse**: the ability to logically appeal an automated decision is perhaps the most important trust-enhancing use of explainable models.

Three transparency concepts

Interpretability: mapping an abstract model concept into an understandable form.

Explainability: interpretability *plus* additional context (stronger requirement).

Contestability (umbrella): constitutional due process principle requires "notice and opportunity to be heard."

Three Goals of Transparency

Your goal determines which method to use. Always evaluate model *performance metrics first*.

Goal 1
Improve the model

Detect "Clever Hans" predictors — models that learned **non-causal shortcuts**. Shortcuts can *increase* accuracy while making the model fragile or biased.

Three Goals of Transparency

Your goal determines which method to use. Always evaluate model *performance metrics first*.

Goal 1: Improve the model

Goal 2
Justify outputs

Different stakeholders need different justifications: developers, deployers, executors, auditors, decision subjects, and data subjects.

Three Goals of Transparency

Your goal determines which method to use. Always evaluate model *performance metrics first*.

Goal 1: Improve the model

Goal 2: Justify outputs

Goal 3
Discover insights

ML models can also reveal relationships between features and outcomes that are not yet known. We cannot target solutions without knowing *why*.

Why Transparency Matters for Alignment

Transparency lets us ask whether the right thing is being optimized in a ML system.

Eight drivers of demand for `transparency` (Molnar, *Interpretable ML*, Ch. 2):

1. Human curiosity and learning.
2. Human desire to find meaning.
3. Knowledge extraction from data.
4. Safety in high-stakes tasks.
5. Bias detection and mitigation.
6. Social acceptance of automated systems.
7. Managing social interaction via shared meaning.
8. Debugging and auditing.

Transparency as a Requirement for Alignment

Alignment implication

Without `transparency` we cannot verify that $h_L^\star \approx h_U^\star$. `Transparency` is a **diagnostic instrument**, not a compliance checkbox.

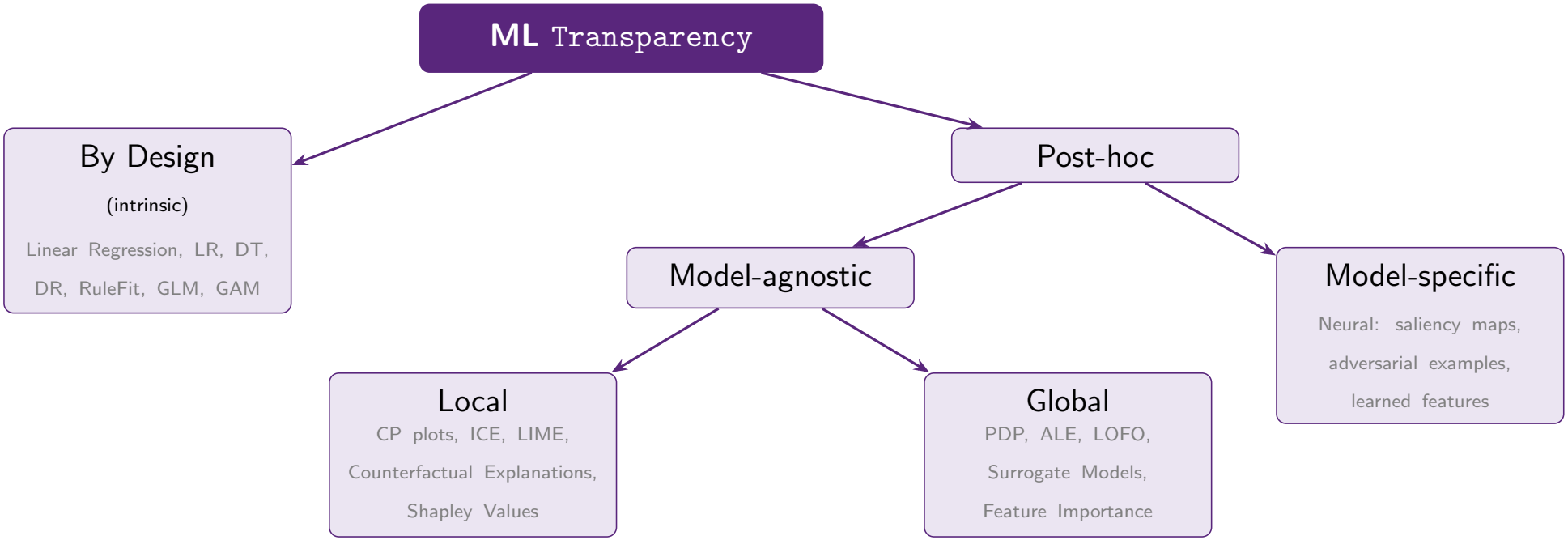
Transparency enables:

- verification of model behavior
- detection of spurious correlations
- debugging unintended optimization

Key Principle

Alignment requires the ability to inspect the reasoning of a model.

Taxonomy of ML Transparency Methods



Today's focus

Today: interpretable-by-design (GLM/GAM) + post-hoc local methods (CP plots, ICE, LIME, Counterfactuals, Shapley Values).

Language: `Explainability` ≡ `Interpretability` ≡ `Contestability`

Things to Consider When Engineering Transparent Models

Feature-level transparency

1. **Explainable feature engineering** — avoid opaque composite features.
2. **Meaningful features** — use variables that stakeholders can reason about.
3. **Monotonic features** — when possible, use features with a monotonic relationship to the target. Binning can induce monotonicity if necessary.

Model-level constraints

1. **Sparsity** (e.g. L1) — fewer parameters, more manageable explanations.
2. **Monotonicity** — positive/negative monotonic constraints prevent illogical behaviour.
3. **Interaction constraints** — limit feature recombination depth.
4. **Linearity & smoothness** — linear functions are monotonic by default; smooth functions can be summarised by derivatives.

Transparent Models by Design

Advantages:

- intrinsic transparency
- simpler reasoning

Examples:

- Logistic Regression
- Decision Trees
- Generalized Linear Models (GLMs)
- Generalized Additive Models (GAMs)

Generalized Linear Models (GLMs)

Standard linear regression:

$$f(x) = \sum_{p=0}^{P} \beta_p x_p + \varepsilon, \quad x_0 = 1 \tag{3.1}$$

Three assumptions often violated: Gaussian outcome, additivity (no interactions), linear feature–outcome relationship.

GLMs relax the distributional assumption via a *link function*:

$$g(\mathbb{E}[Y \mid x]) = \beta_0 + \beta_1 x_1 + \cdots + \beta_P x_P = x^\top \beta \tag{3.2}$$

Generalized Linear Models (GLMs)

Logistic regression (binary outcome):

$$\ln \frac{P(Y{=}1 \mid x)}{1 - P(Y{=}1 \mid x)} = x^\top \beta \tag{3.3}$$

$$\Rightarrow P(Y{=}1) = \frac{1}{1 + \exp(-x^\top \beta)} \tag{3.4}$$

Coefficients β_j: log-odds change per unit increase in x_j, holding others fixed.

Generalized Linear Models (GLMs)

COMPAS connection

In Lecture 01 we ran a logistic regression (LR) on

$$X = \{ \texttt{age}, \texttt{age_cat}, \texttt{sex}, \texttt{race}, \texttt{priors_count}, \texttt{c_charge_degree}, \texttt{score_text}, \texttt{decile_score}, \texttt{days_b_screening_arrest}, \texttt{is_recid}, \texttt{c_jail_in}, \texttt{c_jail_out} \}$$

$$Y = \texttt{two_year_recid} \in \{0, 1\} \quad \text{(recidivism within 2 years)}$$

Each $\hat{\beta}_j$ is directly readable: the model is interpretable by design. The *alignment question* is whether the features X encode proxy discrimination — a **transparency** check, not a performance check.

Generalized Additive Models (GAMs)

Limitation of GLMs: linearity forces the same effect size everywhere. The world is not linear.

GAM replaces $\beta_j x_j$ with a smooth function $f_j(x_j)$:

$$g(\mathbb{E}[Y \mid x]) = \beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_P(x_P)$$

Splines (or other basis functions) learn the f_j from data.

Generalized Additive Models (GAMs)

Why this matters for alignment

- Each f_j can be visualized — a plot of feature j vs. predicted outcome.
- Non-linear but still *additive*: effects of features do not interact (unless interaction terms are added explicitly).
- EBMs (Explainable Boosting Machines) are modern GAMs with pairwise interactions.

Transparency property

Because contributions are additive, we can decompose any prediction exactly:

$$\hat{f}(x) = \beta_0 + \sum_j f_j(x_j) \qquad (3.5)$$

Each term is separately plotted and audited — no black box.

Ceteris Paribus (CP) / Individual Conditional Expectation (ICE)

CP plots visualize how changing *one* feature changes the prediction for **one** data point, holding all others fixed.

ICE plots show *all* CP curves for a dataset in a single plot — one line per instance.

Caveat

Changing one feature while holding others fixed can break real-world dependencies between features.

Algorithm (numerical feature j):

1. Create equidistant grid $z_1, \ldots, z_K$ where $z_1 = \min(\mathbf{x}_j), z_K = \max(\mathbf{x}_j)$.
2. For each grid value: replace x_j with z_k, obtain prediction $\hat{f}(\mathbf{x}^{(i)}_{x_j:=z_k})$.
3. Plot the curve; add a dot for the original data point.

Ceteris Paribus (CP) / Individual Conditional Expectation (ICE)

Algorithm (categorical feature j):

1. Create list of unique categories $z_1, \ldots, z_K$.
2. For each grid value: replace x_j with z_k, obtain prediction $\hat{f}(x^{(i)}_{x_j:=z_k})$.
3. Create bar plot or dot plot with categories. Add a dot for x_j.

Centred ICE (c-ICE): subtract the prediction at an anchor point a (e.g. feature minimum) to remove baseline heterogeneity:

$$\text{ICE}_j^{(i)}(x_j) = \hat{f}(x_j, \mathbf{x}_{-j}^{(i)}) - \hat{f}(a, \mathbf{x}_{-j}^{(i)}) \tag{3.6}$$

Derivative ICE (d-ICE):

1. Assume no interaction between $\mathbf{X}_j$ and other features $\mathbf{X}_{-j}$.
2. $$\begin{aligned}\hat{f}(\mathbf{x}) &= \hat{f}(x_j, \mathbf{x}_C) \\ &= g(x_j) + h(\mathbf{x}_{-j})\end{aligned} \tag{3.7}$$
3. compute $\partial \hat{f} / \partial x_j$ per instance:

$$\frac{\partial \hat{f}(\mathbf{x})}{\partial x_j} = g'(x_j) \tag{3.8}$$

LIME — Local Interpretable Model-agnostic Explanations

Idea: fit a *simple, interpretable* surrogate model g that approximates the black-box $\hat{f}$ *locally* around a single prediction x.

$$\texttt{explanation}(\mathbf{x}) = \arg\min_{g \in \mathcal{G}} \underbrace{L(\hat{f},\, g,\, \pi_{\mathbf{x}})}_{\text{local fidelity}} + \underbrace{\Omega(g)}_{\text{complexity}} \tag{3.9}$$

- $\hat{f}$: original (black-box) model.
- $\mathcal{G}$: is the family of possible explanations
- g: interpretable model (e.g. sparse linear regression).
- $\pi_{\mathbf{x}}$: proximity kernel that defines a neighborhood around $\mathbf{x}$.
- $\Omega(g)$: complexity penalty (prefer fewer parameters).

LIME — Local Interpretable Model-agnostic Explanations

LIME only optimises the loss part; the user controls complexity (number of features K).

LIME for tabular data

1. Sample perturbed instances around x (from training mass centre).
2. Obtain black-box predictions for each.
3. Fit weighted interpretable model.

LIME for text: binary feature per word (1 = present); randomly remove words.

LIME for images: segment into *superpixels*; turn off by replacing with a neutral colour.

Local fidelity $\neq$ global fidelity

LIME is a good approximation *near* x. Do not use LIME outputs to describe global model behaviour.

Counterfactual Explanations (1/3): Concept

A counterfactual explanation describes a causal situation of the form:

"If X had not occurred, Y would not have occurred."

In ML: **the smallest change to feature values that changes a prediction to a predefined output.**

Why counterfactuals?

- **Contrastive** — compare the actual outcome to an alternative, which is how humans naturally reason.
- **Selective** — focus on a small number of feature changes, not the full model internals.
- **Actionable recourse** — tell a subject what they can *do* to change the outcome.

Counterfactual Explanations (1/3): Concept

Example: Peter's loan is rejected.

"If your annual income were $10k higher, the loan would be approved."

Four criteria for a good counterfactual

1. Produces the desired prediction as **closely** as possible.
2. Is as **similar** as possible to the original instance (few feature changes).
3. **Diverse** — multiple viable paths to a different outcome.
4. Feature values in the counterfactual are **plausible** (likely under the data distribution).

Rashomon effect

Multiple counterfactuals can contradict each other. Report all, or define a criterion to select the best.

Counterfactual Explanations (2/3): Wachter et al.

Core idea: minimize a loss that balances prediction fidelity against proximity to the original instance x.

Loss function:

$$L(\mathbf{x}, \mathbf{x}', y', \lambda) = \lambda \cdot \left(\hat{f}(\mathbf{x}') - y'\right)^2 + d(\mathbf{x}, \mathbf{x}') \tag{3.10}$$

- $\mathbf{x}$: original instance
- $\mathbf{x}'$: counterfactual instance to find.
- y': desired (predefined) output.
- λ: balances prediction fidelity vs. proximity.
- d: Manhattan distance weighted by inverse MAD.

Counterfactual Explanations (2/3): Wachter et al.

Distance measure:

$$d(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^{p} \frac{|x_j - x_j'|}{\text{MAD}_j} \tag{3.11}$$

MAD scaling brings all features to the same range and is more robust to outliers than Euclidean distance.

$$\text{MAD}_j = \underset{i \in \{1,\ldots,n\}}{\text{median}} \left(\left| x_j^{(i)} - \underset{l \in \{1,\ldots,n\}}{\text{median}} \left(x_j^{(l)} \right) \right| \right) \tag{3.12}$$

Optimization:

$$\arg\min_{\mathbf{x}'} \max_{\lambda} L(\mathbf{x}, \mathbf{x}', y', \lambda) \tag{3.13}$$

Increase λ until $|\hat{f}(\mathbf{x}') - y'| \leq \varepsilon$ (tolerance criterion).

Counterfactual Explanations (2/3): Wachter et al.

Strengths

- Simple two-term objective, easy to implement.
- Produces a single, proximal counterfactual.
- Model-agnostic.

Limitations

Satisfies only criteria **1** and **2** (proximity + fidelity). Does *not* enforce diversity or plausibility of feature values.

Counterfactual Explanations (3/3): Dandl et al.

Core idea: simultaneously optimize a *four-objective* loss, one objective per quality criterion.

Four-objective loss:

$$L(x, x', y', x^{\text{obs}}) = \left(O_1,\ O_2,\ O_3,\ O_4\right)$$

1. **Prediction fidelity**

$$O_1 = o_1\left(\hat{f}(\mathbf{x}), Y'\right) = \begin{cases} 0 & \text{if } \hat{f}(\mathbf{x}) \in Y' \\ \inf\limits_{y' \in Y'} \left|\hat{f}(\mathbf{x}) - y'\right| & \text{else.} \end{cases} \tag{3.14}$$

2. **Proximity** (Gower distance):

$$O_2 = o_2(\mathbf{x}, \mathbf{x}') = \frac{1}{p} \sum_{j=1}^{p} \delta_G(x_j, x_j') \in [0, 1]. \tag{3.15}$$

Counterfactual Explanations (3/3): Dandl et al.

3. **Sparsity** (L_0 norm — count changed features):

$$O_3 = o_3(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_0 = \sum_{j=1}^{p} \mathbb{I}_{x_j \neq x_j'}. \tag{3.16}$$

4. **Plausibility** (distance to observed data):

$$O_4 = o_4(\mathbf{x}, \mathbf{X}^{\text{obs}}) = \frac{1}{p} \sum_{j=1}^{p} w^{[i]} \sum_{j=1}^{p} \delta_G(x_j, x_j^{[i]}) \in [0, 1] \quad \text{where} \quad \sum_{j=1}^{p} w^{[i]} = 1. \tag{3.17}$$

Counterfactual Explanations (3/3): Dandl et al.

Gower distance handles mixed feature types:

$$\delta_G(x_j, x_j') = \begin{cases} \dfrac{|x_j - x_j'|}{\hat{R}_j} & \text{numerical} \\ \mathbb{I}_{x_j \neq x_j'} & \text{categorical} \end{cases}$$

$\hat{R}_j$ = observed value range of feature j.

Optimization: NSGA-II (Non-dominated Sorting Genetic Algorithm) — a nature-inspired multi-objective solver that applies the principle of "survival of the fittest" to produce a *Pareto front* of diverse counterfactuals.

Advantage over Wachter et al.

All four criteria are optimised simultaneously. The Pareto front gives the decision subject multiple **diverse, plausible** recourse options.

Shapley Values: General Idea

Core framing

A prediction can be explained by treating each feature value as a **player** in a cooperative game where the prediction is the **payout**. Shapley values tell us how to *fairly distribute* the payout among the features.

The game analogy

- **Game**: prediction task for a single instance.
- **Players**: feature values of that instance.
- **Payout**: actual prediction − average prediction across all instances.
- **Coalition**: any subset S of features working together.

Shapley Values: General Idea

Apartment example

A model predicts \$300k for an apartment; the average prediction is \$310k. The difference of $-$\$10k is distributed among `park-nearby`, `cat-banned`, `area-50`, and `floor-2nd`:

- `park-nearby` $\rightarrow$ $+$\$30k
- `area-50` $\rightarrow$ $+$\$10k
- `floor-2nd` $\rightarrow$ \$0
- `cat-banned` $\rightarrow$ $-$\$50k
- **Sum**: $-$\$10k = predicted $-$ average ✓

Key question

How much did each feature value *contribute* to the prediction compared to the **average prediction**? Shapley values answer this for *any* model type.

Shapley Values: Intuition from Linear Models

In a linear model, individual feature effects are directly readable:

$$\hat{f}(\mathbf{x}) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p \tag{3.18}$$

The contribution ϕ_j of feature j is the feature effect minus its average effect:

$$\phi_j(\hat{f}) = \beta_j x_j - \beta_j \mathbb{E}[X_j] \tag{3.19}$$

$\beta_j \mathbb{E}[X_j]$ is the mean effect estimate for feature j.

The contribution is the difference between the feature effect and the average effect.

Shapley Values: Intuition from Linear Models

Summing contributions over all features recovers the full prediction gap:

$$\begin{aligned}\sum_{j=1}^{p} \phi_j(\hat{f}) &= \sum_{j=1}^{p} \left(\beta_j x_j - \beta_j \mathbb{E}[X_j]\right) \\ &= \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right) - \left(\beta_0 + \sum_{j=1}^{p} \beta_j \mathbb{E}[X_j]\right) \\ &= \hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{X})] \qquad (3.20)\end{aligned}$$

This is the predicted value for data point $\mathbf{x}$ minus the average predicted value.

Feature contributions can be negative.

Shapley Values: Intuition from Linear Models

The challenge for black-box models

This decomposition works *only* because linear models have explicit weights β_j. For black-box models such as random forests and neural networks there are no such weights. **Shapley values generalise this idea to any model.**

Shapley Values: Definition

Shapley value of feature j

The **average marginal contribution** of feature j across all possible coalitions $S \subseteq \{1, \ldots, p\} \setminus \{j\}$:

$$\phi_j(val) = \sum_{S \subseteq \{1,\ldots,p\} \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big[val\big(S \cup \{j\}\big) - val(S) \Big] \quad (3.21)$$

Notation guide

- S: coalition — a subset of features *excluding* j.
- $|S|$: size of coalition S.
- p: total number of features.

Shapley Values: Definition

- $\frac{|S|!(p-|S|-1)!}{p!}$: weight equal to the fraction of orderings in which j joins exactly coalition S.
- $val(S \cup \{j\}) - val(S)$: marginal contribution of adding j to S.

Value function $val_{\mathbf{x}}(S)$: the expected prediction for coalition S, marginalising over features *not* in S (denoted $C = \{1, \ldots, p\} \setminus S$), replaced by random draws from the data:

$$val_{\mathbf{x}}(S) = \int \hat{f}(x_1, \ldots, x_p)\, d\mathbb{P}_{X_C} - \mathbb{E}[\hat{f}(\mathbf{X})] \tag{3.22}$$

Interpretation

$\phi_j > 0$: feature j pushed the prediction *above* the average.
$\phi_j < 0$: feature j pushed the prediction *below* the average.

Shapley Values: Axiomatic Properties

Shapley values are the **only** attribution method satisfying all the four axioms that simultaneously define `a fair payout`.

Efficiency

Feature contributions must sum to the prediction gap:

$$\sum_{j=1}^{p} \phi_j = \hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{X})] \tag{3.23}$$

No credit is created or destroyed; the full difference between the prediction and the average is accounted for.

Shapley Values: Axiomatic Properties

Symmetry

If features j and k contribute equally to every possible coalition, i.e.

$$val(S \cup \{j\}) = val(S \cup \{k\}) \quad \forall\, S \subseteq \{1, \ldots, p\} \setminus \{j, k\}$$

then $\phi_j = \phi_k$. Equal players must receive equal payouts.

Dummy

If feature j adds nothing to any coalition, i.e.

$$val(S \cup \{j\}) = val(S) \quad \forall\, S \subseteq \{1, \ldots, p\}$$

then $\phi_j = 0$. A feature that never changes the prediction receives zero attribution.

Shapley Values: Axiomatic Properties

Additivity

For a model whose payout is the sum of two independent games *val* and val^+, the Shapley value for the combined game is:

$$\phi_j^{combined} = \phi_j + \phi_j^+$$

Contributions from independent sub-tasks add up linearly.

Shapley Values: Strengths & Limitations

Strengths

- **Axiomatic fairness**: the only method satisfying Efficiency, Symmetry, Dummy, and Additivity simultaneously.
- **Model-agnostic**: works for any black-box model with no access to internal structure required.
- **Contrastive**: explains each prediction relative to the average, making results easy to communicate to stakeholders.
- **Full attribution**: contributions always sum to the exact prediction gap; no residual is left unaccounted for.
- **Solid theoretical foundation**: grounded in cooperative game theory (Shapley, 1953).

Shapley Values: Strengths & Limitations

Limitations

- **Computational cost**: exact computation is $\mathcal{O}(2^p)$; sampling approximations are required when p is large.
- **Unrealistic instances**: absent features replaced by random draws can produce out-of-distribution combinations, especially when features are correlated.
- **Not a sparse explanation**: all p features receive a Shapley value, unlike LIME which selects a small K.
- **Not causal**: $\phi_j \neq$ the effect of *removing* feature j from the model. Causal claims require a separate causal model.
- **Adversarial risk**: models can be engineered to produce fair-looking Shapley values while discriminating internally (Slack et al., 2020).

Shapley Values: Strengths & Limitations

Governance implication

Shapley values are a rigorous diagnostic tool, but they must be embedded in a documented audit process. A SHAP beeswarm plot is not a substitute for a fairness audit or an actionable recourse mechanism.

Method-specific Limitations of Transparency Methods

ICE / CP

- Feature independence assumption violated in correlated data.
- Visualisation overload with many instances.

LIME

- Neighbourhood definition is unstable; results can change across runs.
- Kernel bandwidth choice is arbitrary.
- Does not guarantee consistency with global model behavior.

Counterfactuals

- May be implausible (Rashomon set problem).
- Multiple counterfactuals can contradict each other.

SHAP

- Computational cost is exponential (approximations needed).
- Marginalisation assumes feature independence.

System-level Limitations of Transparency Methods

- Transparency $\neq$ **ground truth**: post-hoc methods approximate, they do not reveal the true model internals.
- **Explanation washing**: producing explanations to create an *illusion* of transparency without meaningful recourse.
- **Stakeholder mismatch**: a SHAP beeswarm plot is not an explanation for a loan applicant.
- **Adversarial explanations**: models can be constructed to produce "fair-looking" SHAP plots while discriminating internally (Slack et al. 2020).
- **Clever Hans problem**: interpretability reveals shortcuts; it does not automatically remove them.

Governance implication: Transparency tools must be part of a **documented audit process**, not a one-time report.

Live Coding: Session Plan

Continuing the COMPAS recidivism example from Lecture 01.

Learning objectives

1. Fit a logistic regression (GLM) and a gradient-boosted tree (black-box) on the cleaned COMPAS cohort.
2. Generate LIME explanations for individual predictions.
3. Compute SHAP values and plot a beeswarm + waterfall.
4. Generate counterfactuals using `DiCE`.
5. Compare explanations across a Black defendant and a White defendant with similar risk scores.

Source code for today: R code from Lecture 1. You are expected to convert this to a working Python code.

Live Coding: COMPAS Explainability Pipeline

Step 0 — Train / test split

```
from sklearn.model_selection import train_test_split

# X and y from the cleaned COMPAS cohort (Lecture 01)

X = df[features]
y = df[target].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Train: {X_train.shape}, Test: {X_test.shape}")
```

Step 1 — Fit logistic regression and gradient-boosted tree

Live Coding: COMPAS Explainability Pipeline

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

numeric_features & category_features as in Lecture 01.

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(
                handle_unknown="ignore"), category_features),
])

# Logistic regression (GLM -- interpretable by design)
lr_pipeline = Pipeline([
```

Live Coding: COMPAS Explainability Pipeline

```
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000,
                                      random_state=42))
])
lr_pipeline.fit(X_train, y_train)

# Gradient-boosted tree (black-box)
gbt_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", GradientBoostingClassifier(
                        n_estimators=200,
                        max_depth=4,
                        random_state=42))
])
gbt_pipeline.fit(X_train, y_train)
```

Live Coding: COMPAS Explainability Pipeline

Step 2 — Compare model performance by race

```
import pandas as pd
from sklearn.metrics import (confusion_matrix,
                             accuracy_score,
                             roc_auc_score)

def group_metrics(pipeline, X, y, group_col="race"):
    results = []
    X_ = X.copy()
    X_["actual"] = y.values
    X_["pred"] = pipeline.predict(X)
    X_["pred_prob"] = pipeline.predict_proba(X)[:, 1]

    for group, gdf in X_.groupby(group_col):
        tn, fp, fn, tp = confusion_matrix(
            gdf["actual"], gdf["pred"]
```

Live Coding: COMPAS Explainability Pipeline

```
        ).ravel()
        results.append({
            "race" : group,
            "n" : len(gdf),
            "accuracy" : round(accuracy\_score(
                            gdf["actual"], gdf["pred"]), 3),
            "FPR" : round(fp / (fp + tn), 3),
            "FNR" : round(fn / (fn + tp), 3),
            "AUC" : round(roc_auc_score(
                            gdf["actual"],
                            gdf["pred_prob"]), 3),
        })
    return pd.DataFrame(results).sort_values("n", ascending=False
        )

print(" Logistic Regression ")
```

Live Coding: COMPAS Explainability Pipeline

```
print(group_metrics(lr_pipeline, X_test, y_test))

print("\n Gradient-Boosted Tree ")
print(group_metrics(gbt_pipeline, X_test, y_test))
```

Step 3 — LIME explanations: Black vs White defendant

```
# !pip install lime
from lime.lime_tabular import LimeTabularExplainer
import numpy as np

X_train_enc = preprocessor.fit_transform(X_train)

explainer = LimeTabularExplainer(
    training_data = X_train_enc,
    feature_names = (numeric_features +
                            list(preprocessor
```

Live Coding: COMPAS Explainability Pipeline

```
                                .named_transformers_["cat"]
                                .get_feature_names_out(
                                    category_features))),
    class_names = ["No Recid", "Recid"],
    mode = "classification",
    discretize_continuous= True,
    random_state = 42
)

# Select a Black and White defendant with similar predicted risk
X_test_prob = gbt_pipeline.predict_proba(X_test)[:, 1]
X_test_copy = X_test.copy()
X_test_copy["pred_prob"] = X_test_prob

black_idx = X_test_copy[
    X_test_copy["race"] == "African-American"
```

Live Coding: COMPAS Explainability Pipeline

```
].sort_values("pred_prob").index[len(X_test_copy) // 2]

white_idx = X_test_copy[
    X_test_copy["race"] == "Caucasian"
].sort_values("pred_prob").index[len(X_test_copy) // 2]

# Generate and display LIME explanations
for label, idx in [("Black defendant", black_idx),
                   ("White defendant", white_idx)]:
    row_enc = preprocessor.transform(X_test.loc[[idx]])
    exp = explainer.explain_instance(
        data_row = row_enc.toarray()[0],
        predict_fn= gbt_pipeline["classifier"].predict_proba,
        num_features=6
    )
    print(f"\n LIME: {label} ")
```

Live Coding: COMPAS Explainability Pipeline

```
    print(f"Predicted probability: "
          f"{X_test_prob[X_test.index.get_loc(idx)]:.3f}")
    for feat, weight in exp.as_list():
        print(f" {feat:<40} {weight:+.4f}")
```

Step 4 — SHAP beeswarm and waterfall: Black vs White

```
# !pip install shap
import shap
import matplotlib.pyplot as plt

X_test_enc = preprocessor.transform(X_test)
feat_names = (numeric_features +
              list(preprocessor
                   .named_transformers_["cat"]
                   .get_feature_names_out(category_features)))
```

Live Coding: COMPAS Explainability Pipeline

```
explainer_shap = shap.Explainer(
    gbt_pipeline["classifier"],
    shap.maskers.Independent(X_test_enc, max_samples=100)
)
shap_values = explainer_shap(X_test_enc)

# Global beeswarm
shap.summary_plot(shap_values, X_test_enc,
                  feature_names=feat_names)

# Waterfall: Black defendant
black_loc = X_test.index.get_loc(black_idx)
white_loc = X_test.index.get_loc(white_idx)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, loc, label in [
```

Live Coding: COMPAS Explainability Pipeline

```
    (axes[0], black_loc, "Black defendant"),
    (axes[1], white_loc, "White defendant")
]:
    plt.sca(ax)
    shap.waterfall_plot(shap_values[loc],
                        max_display=8,
                        show=False)
    ax.set_title(label)

plt.tight_layout()
plt.show()
```

Step 5 — Counterfactuals with DiCE: Black vs White

Live Coding: COMPAS Explainability Pipeline

```
# !pip install dice-ml
import dice_ml
from dice_ml import Dice

# DiCE requires a pandas DataFrame and a wrapped model
dice_data = dice_ml.Data(
    dataframe = pd.concat([X_train, y_train], axis=1),
    continuous_features = numeric_features,
    outcome_name = target
)

dice_model = dice_ml.Model(
    model = gbt_pipeline,
    backend = "sklearn",
    model_type = "classifier"
)
```

Live Coding: COMPAS Explainability Pipeline

```
exp_dice = Dice(dice_data, dice_model, method="random")

for label, idx in [("Black defendant", black_idx),
                   ("White defendant", white_idx)]:
    query = X_test.loc[[idx]]
    cf = exp_dice.generate_counterfactuals(
        query,
        total_CFs = 3,
        desired_class = "opposite",
        permitted_range = {"age" : [18, 70],
                           "priors_count": [0, 38]},
        features_to_vary = ["age", "priors_count",
                            "c_charge_degree"]
    )
    print(f"\n DiCE counterfactuals: {label} ")
```

Live Coding: COMPAS Explainability Pipeline

```
cf.visualize_as_dataframe(show_only_changes=True)
```

Homework: Due at 12pm ET on March 30, 2026

Group Homework 2 — Mathematical: Shapley Axioms and Fairness
Let $\hat{f}$ be a binary classifier and ϕ_j the Shapley value of feature j (race) for instance x.

a. State the **efficiency** axiom and explain why it implies $\phi_{\text{race}} = 0$ if and only if race contributes nothing *beyond the average prediction*, not if and only if the model is "race-blind."

b. Construct a simple two-feature example ($x_1 = \text{race}$, $x_2 = \text{prior arrests}$) where $\phi_{\text{race}} = 0$ yet the model violates equal FPR parity. Interpret this result in the context of the COMPAS case.

c. Discuss: is $\phi_{\text{race}} = 0$ a sufficient condition for fairness? Under what governance framework would you additionally require $\phi_{\text{race}} = 0$ *and* FPR parity?

Group Homework: Due at 12pm ET on March 30, 2026

Individual Homework 2 — Coding: Explaining the COMPAS Replacement Model

Building on the live coding exercise in Lecture 02:

1. Compute **SHAP values** for your model on the test set. Produce: (a) a beeswarm summary plot, (b) a waterfall plot for the highest-risk and lowest-risk defendant in each racial group.
2. Run **LIME** on the same four individuals. Compare LIME and SHAP feature attributions. Where do they agree? Where do they diverge? What does divergence imply for governance?
3. Generate at least one **counterfactual** per individual using DiCE (Python) or `NICE` (R). Report the minimal feature changes required to flip the prediction. Flag any counterfactuals that require changes to *immutable* features (race, sex).

Group Homework: Due at 12pm ET on March 30, 2026

4. Write a 300-word **governance memo** addressed to a hypothetical court auditor: summarise what your explanations reveal about model behaviour, any limitations of the methods, and what further monitoring you recommend.

Submission Requirements: Follow the same procedure used for homework 1.

Five Core Takeaways

1. `Transparency` $\neq$ **trust.** Greater understanding of a poorly built model can decrease trust — and should.
2. `Transparency` **is diagnostic, not cosmetic.** Use it to detect Clever Hans predictors, proxy features, and alignment gaps.
3. **No single method is sufficient.** LIME, SHAP, and counterfactuals answer different questions and have different failure modes. Use them together.
4. **Stakeholder context determines the right explanation.** A SHAP waterfall is not an explanation for a loan applicant; a counterfactual is.
5. `Transparency` **without governance is explanation washing.** Methods must feed into documented audit processes, recourse mechanisms, and monitoring workflows.

Readings (for this lecture)

1. **Required**
   1. Molnar, C., *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 3rd ed. Chapters 2, 3, 4, 8, 12–15, 17.
   2. Hall, P. & Gill, N., *Machine Learning for High-Risk Applications*, O'Reilly, 2023. Chapter 2.
2. **Optional**
   1. Slack, D. et al. (2020). "Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods." *AIES 2020*.
   2. Wachter, S., Mittelstadt, B. & Russell, C. (2017). "Counterfactual Explanations Without Opening the Black Box: Automated Decisions and the GDPR." *Harvard Journal of Law & Technology*, 31(2).
   3. Lundberg, S. & Lee, S.-I. (2017). "A Unified Approach to Interpreting Model Predictions." *NeurIPS 2017*.

Thank You!

Any questions?

s.akinwumi@gwu.edu

Office hours: Funger Hall 415R, Fri 5:00–7:00 PM

Math Terms

Math terms used throughout this lecture notes:

Math Term	Meaning
X_j	Random variable corresponding to feature j.
X	Set of all random variables.
$\mathbb{P}$	Probability measure.
$\mathbb{P}(X_j = c)$	Probability that random variable X_j takes on the value c.
x_j	Vector of values for feature j across multiple data instances.
$\mathbf{x}^{(i)}$	Feature vector of the i-th instance.
$x_j^{(i)}$	Value of feature j for the i-th instance.
$y^{(i)}$	Target value for the i-th instance.
$\mathbf{X}$	Feature matrix, where rows are instances and columns are features.
$\mathbf{y}$	Vector of target values, one for each instance.

Math Terms cont'd

Math Term	Meaning
n	Number of data instances (rows in $\mathbf{X}$).
p	Number of features (columns in $\mathbf{X}$).

Grading and Policy

- Grading
  - Weekly assignment: $\frac{2.5}{10}$
  - Public GitHub with working codebase: $\frac{2.5}{10}$
  - Weekly quiz: $\frac{2}{10}$
  - Capstone: $\frac{3}{10}$
- Coding/Scripting
  - Python
  - R
- Collaborative Infrastructure
  - GitHub

Grading and Policy

- Capstone project
  - 2024 Home Mortgage Disclosure Act (HMDA): Loan/Application Records (LAR) data
  - Public LAR data: dictionary
  - Goal: Develop a classification model that predicts `action_taken` and apply principles taught in class to the model.
  - Label creation: `action_taken` values 1 and 2 coded as 1 and 3 coded as 0 while records matching other values are filtered out.
  - Group (no more than 5 members and no less than 3 members)
  - Group members are randomly assigned by instructor using the standard *math* and *random* libraries in **Python**.
- Syllabus: link
- Officer hours: Funger Hall 415R, Fri, 4:00PM - 6:00PM
- Class resources:
  - Instructor's materials
  - Additional resources from Patrick Hall